

ВАЛИДАЦИЯ НА Email, парола, EGN

Преговорни забележки: понеже конструкторите могат да имат по няколко презареждания overloads, по подразбиране е този, който не изисква параметри, т.е. между скобите му е празно (). Винаги е по-добре да има конструктор по подразбиране за всеки случай – слагайте го! Също, конструкторът не връща тип.

Regex = regular expression = регулярен израз



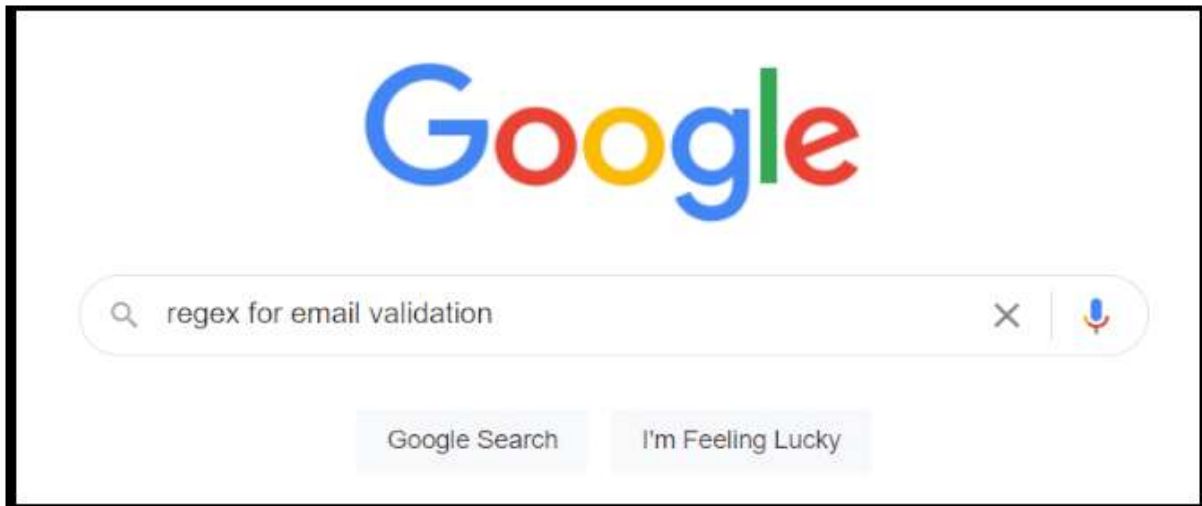
В случая е синтаксис за валидация на стар тип имейл:

букви + @ букви + . разширение с дължина от 2 до 4 като .bg, .com и т.н.

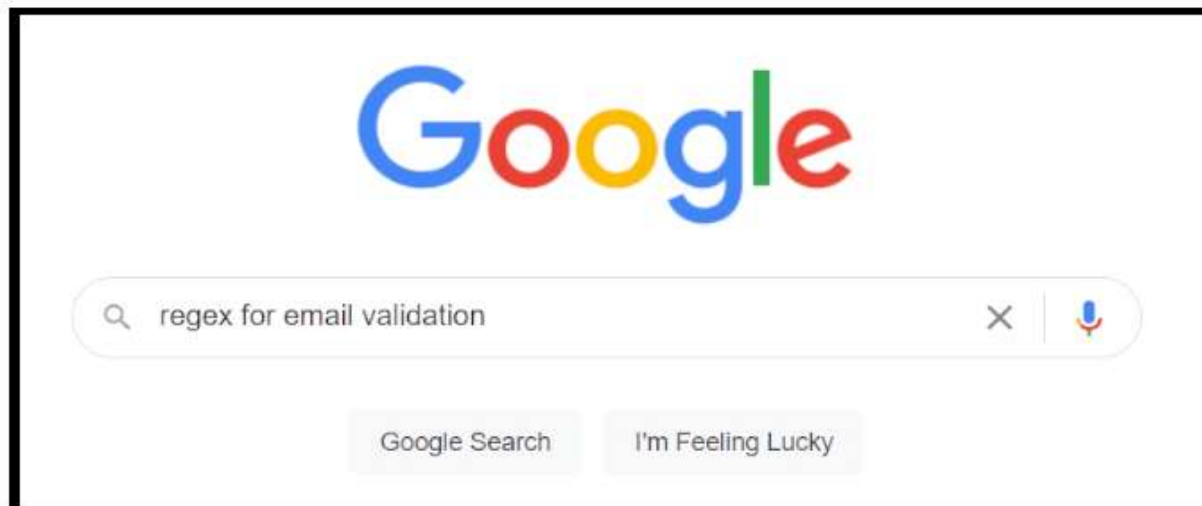
Напомням, че \ е escape-последователност, т.е. \. = точка, т.е. дословно четене

При новите имейли вече имаме разширение с различна дължина и този regex отпада

DAY1 OF PROGRAMMING



10 YEARS OF PROGRAMMING



Забележка: gmail игнорира точките в username – username@gmail.com и user.name@gmail.com се считат за един и същи.

Може да разгледаме как се прави валидация на email, парола и ЕГН с помощта на DeepSeek.

Като чат-бот, DeepSeek е по-специализиран към математика и програмиране.

```

12 private string email;
13 private string password;
14 private string egn;
15
16 public string Email...
30
31 public string Password...
46
47 public string EGN...
61
62 public User(string email, string password, string egn) // конструктор...
68
69 private bool IsValidEmail(string email)...
81
82 private (bool IsValid, string ErrorMessage) ValidatePassword(string password)...
101
102 private bool IsValidEGN(string egn)...
132
133 private string HashPassword(string password) // демонстрация на хеширане на парола...
138
139 public bool VerifyPassword(string inputPassword)...
144
145 public override string ToString()...

```

Annotations in the image:

- private fields** (next to lines 12-14)
- public properties + validations get, set** (next to lines 16-17 and 31-32)
- public ctor (parameters)** (next to line 62)
- ctor + Tab + Tab** (next to line 62)
- methods** (next to line 69)
- constructor is before personal methods** (next to line 62)
- винаги първо private, после public като подредба** (next to line 12)
- 1 reference** (next to line 16)
- 2 references** (next to line 31)
- 1 reference** (next to line 47)
- 0 references** (next to line 62)
- 1 reference** (next to line 69)
- 1 reference** (next to line 82)
- 1 reference** (next to line 102)
- 2 references** (next to line 133)
- 0 references** (next to line 139)
- 2** (next to line 145)

По същия начин, както свойствата можехме да ги пишем с **prop + Tab + Tab**, можем да допишем и конструктор с **ctor + Tab + Tab**

Нека разгледаме подробно:

```

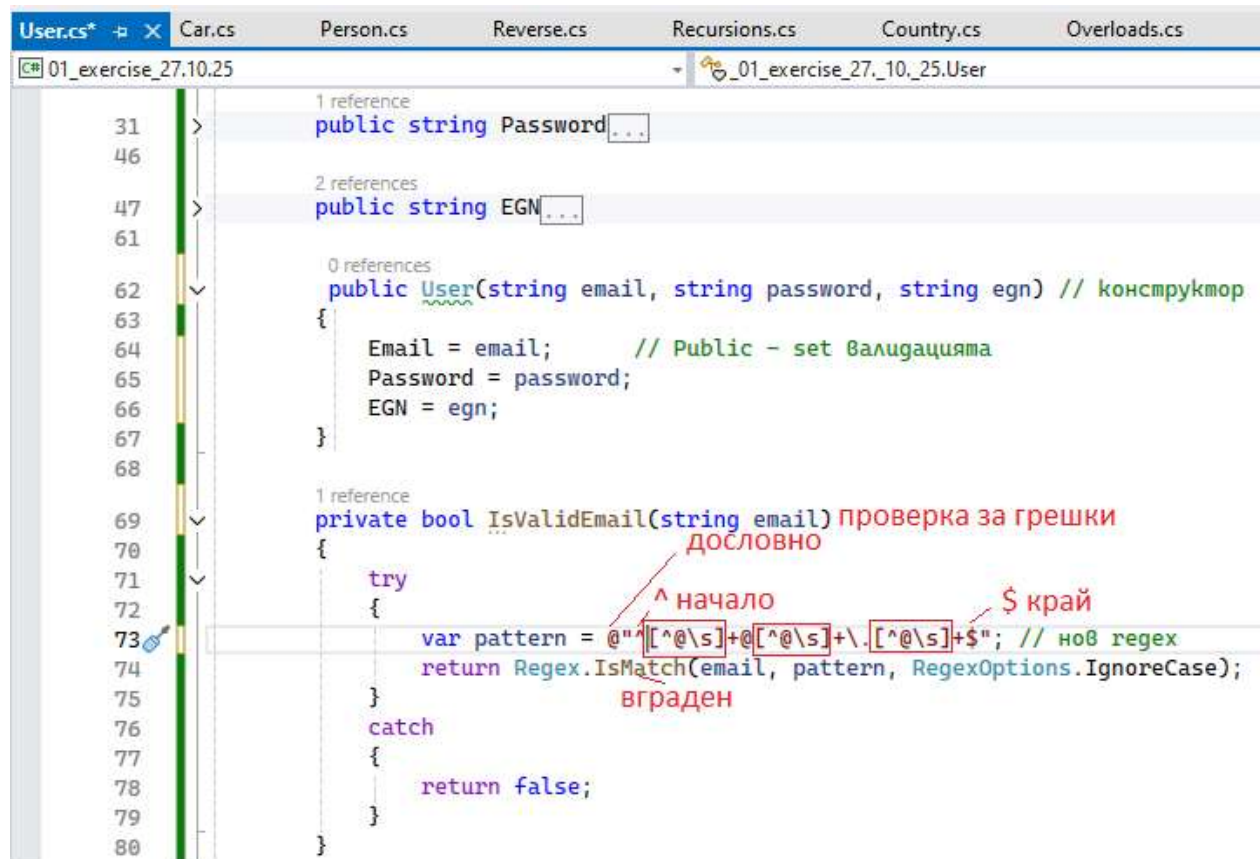
10 class User
11 {
12     private string email;
13     private string password;
14     private string egn;
15
16     public string Email
17     {
18         get { return email; }
19         set
20         {
21             if (string.IsNullOrEmpty(value))
22                 throw new ArgumentException("Email не може да бъде празен!");
23
24             if (!IsValidEmail(value)) ние ще напишем метода
25                 throw new ArgumentException("Невалиден email формат!");
26
27             email = value.Trim().ToLower();
28         }
29     }

```

Annotations in the image:

- 1 reference** (next to line 10)
- 2 references** (next to line 16)
- вграден** (next to line 20)
- ниие ще напишем метода** (next to line 24)

Като конструкция в set е по-добре да има първо проверка АКО е грешно с **!... (value)** – CW а в противен случай да се приеме стойността **= value**



```
1 reference
public string Password...

2 references
public string EGN...

0 references
public User(string email, string password, string egn) // конструктор
{
    Email = email; // Public - set валидацията
    Password = password;
    EGN = egn;
}

1 reference
private bool IsValidEmail(string email) проверка за грешки
{
    try
    {
        var pattern = @"^[^@\\s]+@^[^@\\s]+\\.^[^@\\s]+$"; // ноб regex
        return Regex.IsMatch(email, pattern, RegexOptions.IgnoreCase);
    }
    catch
    {
        return false;
    }
}
```

^ начало
\$ край
дословно
ноб regex
вграден

Нека разгледаме сега валидация на парола:

```
User.cs*  Car.cs  Person.cs  Reverse.cs  Recursions.cs  Country.cs  Overloads.cs  Program
01_exercise_27.10.25  _01_exercise_27_10_25.User

31 1 reference
32 public string Password
33 {
34     get { return "*****"; } // Никога не връщаме реалната парола
35     set
36     {
37         if (string.IsNullOrEmpty(value))
38             throw new ArgumentException("Паролата не може да бъде празна!");
39         var validationResult = ValidatePassword(value);
40         if (!validationResult.IsValid)
41             throw new ArgumentException(validationResult.ErrorMessage);
42         password = HashPassword(value); // Хеширане на паролата
43     }
44 }
45
46
47 2 references
48 public string EGN[...]
49
50
51
52 0 references
53 public User(string email, string password, string egn) // конструктор[...]
54
55
56
57 1 reference
58 private bool IsValidEmail(string email)[...]
```

```
1 reference
private (bool IsValid, string ErrorMessage) ValidatePassword(string password)
{
    // наш метод (input variable)

    if (password.Length < 8)
        return (false, "Паролата трябва да е поне 8 символа!");

    if (!password.Any(char.IsUpper))
        return (false, "Паролата трябва да съдържа поне една главна буква!");

    if (!password.Any(char.IsLower))
        return (false, "Паролата трябва да съдържа поне една малка буква!");

    if (!password.Any(char.IsDigit))
        return (false, "Паролата трябва да съдържа поне една цифра!");
    // lambda expression - търсачка

    if (!password.Any(ch => !char.IsLetterOrDigit(ch)))
        return (false, "Паролата трябва да съдържа поне един специален символ!");

    return (true, "");
}
```

Изписва повече от 1 съобщения при грешка

```
User.cs*  Car.cs  Person.cs  Reverse.cs  Recursions.cs  Country.cs  Overloads.cs  Program.cs*  GCI
01_exercise_27.10.25  01_exercise_27.10.25.User  VerifyPass

133 private string HashPassword(string password) // демонстрация на хеширане на парола
134 {
135     // В реална система използваме BCrypt или подобна библиотека
136     return Convert.ToBase64String(System.Text.Encoding.UTF8.GetBytes(password + "SALT"));
137 }
138
139 public bool VerifyPassword(string inputPassword)
140 {
141     string hashedInput = HashPassword(inputPassword);
142     return password == hashedInput;
143 }
```

ЕГН:

```
User.cs*  Car.cs  Person.cs  Reverse.cs  Recursions.cs  Country.cs  Overloads.cs
01_exercise_27.10.25  01_exercise_27.10.25.User

47 public string EGN
48 {
49     get { return egn; }
50     set
51     {
52         if (string.IsNullOrWhiteSpace(value))
53             throw new ArgumentException("ЕГН не може да бъде празно!");
54         if (!IsValidEGN(value))
55             throw new ArgumentException("Невалидно ЕГН!");
56         egn = value.Trim();
57     }
58 }
59
60 }
```

```

User.cs*  Car.cs  Person.cs  Reverse.cs  Recursions.cs  Country.cs  Overloads.cs  P
01_exercise_27.10.25  - 01_exercise_27_10_25.User

1 reference
102 private bool IsValidEGN(string egn)
103 {
104     egn = egn.Trim();
105
106     if (egn.Length != 10) return false;
107
108     if (!egn.All(char.IsDigit)) return false;
109     ПОДСТРИНГ ОТ, ДО
110     int year = int.Parse(egn.Substring(0, 2)); // части от ЕГН
111     int month = int.Parse(egn.Substring(2, 2));
112     int day = int.Parse(egn.Substring(4, 2));
113
114     if (month >= 41 && month <= 52) month -= 40; // слег 1999
115
116     if (month < 1 || month > 12 || day < 1 || day > 31) return false;
117
118     int[] weights = { 2, 4, 8, 5, 10, 9, 7, 3, 6 }; // контролната цифра
119     int sum = 0;
120
121     for (int i = 0; i < 9; i++)
122     {
123         sum += int.Parse(egn[i].ToString()) * weights[i];
124     }
125
126     int remainder = sum % 11; съкратен if - else
127     int controlDigit = remainder == 10 ? 0 : remainder;
128     int actualControlDigit = int.Parse(egn[9].ToString());
129
130     return controlDigit == actualControlDigit;
131 }

```

Последно:

```

0 references
145 public override string ToString()
146 {
147     return $"Email: {Email}, EGN: {EGN}";
148 }

```

```

class User
{
    private string email;
    private string password;
    private string egn;

    public string Email
    {
        get { return email; }
        set
        {
            if (string.IsNullOrEmpty(value))
                throw new ArgumentException("Email не може да бъде празен!");

            if (!IsValidEmail(value))
                throw new ArgumentException("Невалиден email формат!");
        }
    }
}

```

```

        email = value.Trim().ToLower();
    }
}

public string Password
{
    get { return "*****"; } // Никога не връщаме реалната парола
    set
    {
        if (string.IsNullOrEmpty(value))
            throw new ArgumentException("Паролата не може да бъде празна!");

        var validationResult = ValidatePassword(value);
        if (!validationResult.IsValid)
            throw new ArgumentException(validationResult.ErrorMessage);

        password = HashPassword(value); // Хеширане на паролата
    }
}

public string EGN
{
    get { return egn; }
    set
    {
        if (string.IsNullOrEmpty(value))
            throw new ArgumentException("ЕГН не може да бъде празно!");

        if (!IsValidEGN(value))
            throw new ArgumentException("Невалидно ЕГН!");

        egn = value.Trim();
    }
}

public User(string email, string password, string egn) // конструктор
{
    Email = email; // Public - set валидацията
    Password = password;
    EGN = egn;
}

private bool IsValidEmail(string email)
{
    try
    {
        var pattern = @"^[^@\s]+@[^@\s]+\.[^@\s]+$"; // нов regex
        return Regex.IsMatch(email, pattern, RegexOptions.IgnoreCase);
    }
    catch
    {
        return false;
    }
}

private (bool IsValid, string ErrorMessage) ValidatePassword(string
password)

```



```

    {
        if (password.Length < 8)
            return (false, "Паролата трябва да е поне 8 символа!");

        if (!password.Any(char.IsUpper))
            return (false, "Паролата трябва да съдържа поне една главна
буква!");

        if (!password.Any(char.IsLower))
            return (false, "Паролата трябва да съдържа поне една малка буква!");

        if (!password.Any(char.IsDigit))
            return (false, "Паролата трябва да съдържа поне една цифра!");

        if (!password.Any(ch => !char.IsLetterOrDigit(ch)))
            return (false, "Паролата трябва да съдържа поне един специален
символ!");

        return (true, "");
    }

private bool IsValidEGN(string egn)
{
    egn = egn.Trim();

    if (egn.Length != 10) return false;

    if (!egn.All(char.IsDigit)) return false;

    int year = int.Parse(egn.Substring(0, 2)); // части от ЕГН
    int month = int.Parse(egn.Substring(2, 2));
    int day = int.Parse(egn.Substring(4, 2));

    if (month >= 41 && month <= 52) month -= 40; // след 1999

    if (month < 1 || month > 12 || day < 1 || day > 31) return false;

    int[] weights = { 2, 4, 8, 5, 10, 9, 7, 3, 6 }; // контролната цифра
    int sum = 0;

    for (int i = 0; i < 9; i++)
    {
        sum += int.Parse(egn[i].ToString()) * weights[i];
    }

    int remainder = sum % 11;
    int controlDigit = remainder == 10 ? 0 : remainder;
    int actualControlDigit = int.Parse(egn[9].ToString());

    return controlDigit == actualControlDigit;
}

private string HashPassword(string password) // демонстрация на хеширане на
парола
{
    // В реална система използвайте BCrypt или подобна библиотека
    return
Convert.ToBase64String(System.Text.Encoding.UTF8.GetBytes(password + "SALT"));

```

```
}

public bool VerifyPassword(string inputPassword)
{
    string hashedInput = HashPassword(inputPassword);
    return password == hashedInput;
}

public override string ToString()
{
    return $"Email: {Email}, EGN: {EGN}";
}
}
```